

Lab 1

Introduction to Assembly

Prof. Charan Bhaskar

This guide describes the basics of 32-bit x86 assembly language programming, covering a small but useful subset of the available instructions and assembler directives. There are several different assembly languages for generating x86 machine code. The one we will use in CMPE127 is the Microsoft Macro Assembler (MASM) assembler. MASM uses the standard Intel syntax for writing x86 assembly code.

The full x86 instruction set is large and complex (Intel's x86 instruction set manuals comprise over 2900 pages), and we do not cover it all in this guide. For example, there is a 16-bit subset of the x86 instruction set. Using the 16-bit programming model can be quite complex. It has a segmented memory model, more restrictions on register usage, and so on. In this guide, we will limit our attention to more modern aspects of x86 programming, and delve into the instruction set only in enough detail to get a basic feel for x86 programming.

Registers

Registers are small, fast storage locations within the CPU that are used for temporary data storage, manipulation, and control during program execution. Modern (i.e 386 and beyond) x86 processors have eight 32-bit general purpose registers, as depicted in Figure 1. The register names are mostly historical. For example, EAX used to be called the accumulator since it was used by a number of arithmetic operations, and ECX was known as the counter since it was used to hold a loop index. Whereas most of the registers have lost their special purposes in the modern instruction set, by convention, two are reserved for special purposes — the stack pointer (ESP) and the base pointer (EBP). The x86 architecture includes several types of registers, each serving a specific purpose:

1. **General purpose registers:** General-purpose registers are a set of storage locations within a CPU that are designed to hold data temporarily during program execution. These registers can be used for a wide range of purposes, including arithmetic and logic operations, data manipulation, memory access, and more. In the x86 architecture, these registers are the workhorses of the CPU, allowing it to perform various tasks efficiently.
 - a. EAX (RAX in 64-bit mode): Accumulator: Used for arithmetic operations and storing intermediate results.

```
mov eax, 5    ; EAX = 5
add eax, 7    ; EAX = EAX + 7 = 12
```
 - b. EBX (RBX in 64-bit mode): Base: Typically used as a pointer for memory access or data storage.

```
mov ebx, 2    ; EBX = index of array element
mov eax, [array + ebx * 4] ; Load array[index] into EAX
```

- c. ECX (RCX in 64-bit mode): Counter: Often used in loop control and string manipulation.

```
mov ecx, 0    ; Initialize loop counter
loop_start:   ; Loop body
    inc ecx    ; Increment counter
    cmp ecx, 10 ; Compare with 10
    jl loop_start ; Jump if less than 10
```

- d. EDX (RDX in 64-bit mode): Data: Used for I/O operations, division, and certain control operations.

```
mov eax, 23    ; Numerator
mov edx, 5      ; Denominator
div edx        ; Divide EAX by EDX, quotient in EAX, remainder in EDX
```

- e. ESI (RSI in 64-bit mode): Source Index: Typically used as a source index in data copying.

- f. EDI (RDI in 64-bit mode): Destination Index: Typically used as a destination index in data copying.

```
mov esi, source_array ; Source array pointer
mov edi, dest_array    ; Destination array pointer
mov ecx, array_size    ; Number of elements
rep movsd              ; Copy ECX doublewords from ESI to EDI
```

- g. EBP (RBP in 64-bit mode): Base Pointer: Used to reference function parameters and local variables on the stack.

```
mov ebp, esp    ; Set up base pointer
mov eax, [ebp + 8] ; Access function's first parameter
sub eax, [ebp - 4] ; Subtract local variable value
```

- h. ESP (RSP in 64-bit mode): Stack Pointer: Points to the top of the stack, used for function call management and local variable storage.

```
push eax    ; Push EAX onto the stack
pop ebx     ; Pop value from the stack into EBX
```

For the EAX, EBX, ECX, and EDX registers, subsections may be used. For example, the least significant 2 bytes of EAX can be treated as a 16-bit register called AX. The least significant byte of AX can be used as a single 8-bit register called AL, while the most significant byte of AX can be used as a single 8-bit register called AH. These names refer to the same physical register. When a two-byte quantity is placed into DX, the update affects the value of DH, DL, and EDX. These sub-registers are mainly hold-overs from older, 16-bit versions of the instruction set. However, they are sometimes convenient when dealing with data that are smaller than 32-bits (e.g. 1-byte ASCII characters).

When referring to registers in assembly language, the names are not case-sensitive. For example, the names EAX and eax refer to the same register.

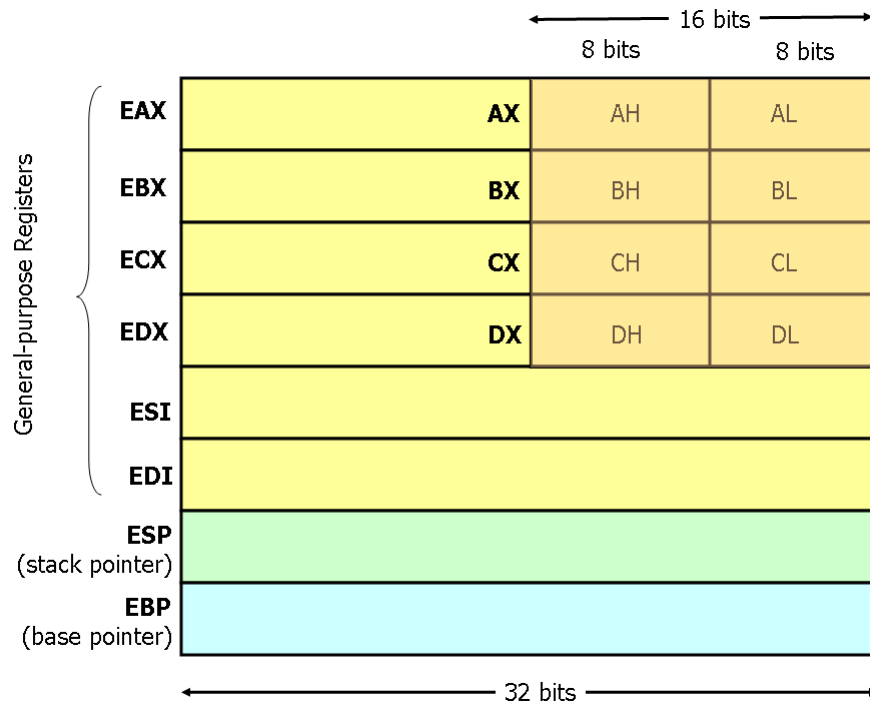


Fig 1: 8086 Registers

- 2. Segment Registers:** These registers are used to store segment information for memory addressing. They are largely obsolete in modern operating systems and protected mode.
- CS (Code Segment):** Used to store the segment address for the code segment. It indicates the segment where the currently executing program code resides.
 - DS (Data Segment):** Used to store the segment address for the data segment. It indicates the segment where data for general-purpose use is stored.
 - SS (Stack Segment):** Used to store the segment address for the stack segment. It indicates the segment where the program's stack is located.
 - ES (Extra Segment):** Historically used for additional data storage, similar to the DS register.
 - FS and GS:** Additional segment registers introduced in later x86 architectures (80386 and later). These were designed to provide additional segment pointers, often used for special purposes like thread-local storage.
- Flags Register (EFLAGS or RFLAGS):** This register holds various status flags that are modified by arithmetic and logical operations. It includes flags like carry, zero, sign, overflow, and more.

- 3. Instruction Pointer (EIP or RIP):** This register points to the memory address of the next instruction to be executed.

Variables: In assembly language programming, variables are stored in memory using memory addresses. Each variable is associated with a specific memory location, which can be referenced by its address. When you declare a variable in assembly language, you're essentially reserving a specific amount of memory to hold its value. This memory allocation is typically done in the .data section of your program. Every byte in a computer's memory has a unique address. These addresses are numerical values that represent the location of each byte in memory. Variables are stored at specific memory addresses. The DS register often points to the start of the data segment where variables are stored. To refer to the address of a memory location held by a variable use &variable-name. For example &sum

Endianness: Endianness refers to the byte order in which multi-byte data types are stored in computer memory. There are two main types of endianness: Big-Endian and Little-Endian. In Big-Endian, the most significant byte of a multi-byte value is stored at the lowest memory address, while in Little-Endian, the least significant byte is stored at the lowest address. This distinction becomes crucial when data is shared between systems with different endianness, as it can lead to data interpretation errors. Many modern processors, including x86 and ARM, use Little-Endian by default due to its potential for better memory access performance.



Fig 2: Endianness

Lab Work

- Make a new project as demonstrated in the demo and copy the following code to the source file. Configure it for MASM and run it.

```
;Lab1 source code

.386
.model flat, stdcall
.stack 4096
ExitProcess proto,dwExitCode:dword

.data
```

```
a            WORD    9
sum          DWORD   ?
diff         DWORD   ?

.code
main proc
    mov eax, 7
    add eax, 4
    mov sum, eax

    mov eax, 12345
    sub eax, 2
    mov diff, eax

    invoke ExitProcess, 0

main endp
end main
```

- Check if the memory format it is using is big endian or little endian. Attach screenshot of memory window. (Hint: You can check the value of diff after loading the difference)
- Write your understanding of the code.